# Software Engineering

## UML Klassendiagramm

Prof. Dr.-Ing. Sven Graupner

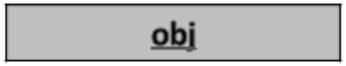


# Gliederung

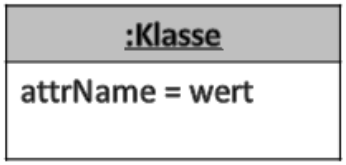
1. Objekte und Klassen
   - Objekt, Attribute, Methoden
2. Schnittstellen (Interfaces)
3. Beziehungen zwischen Klassen
   - Generalisierung, Assoziationsklasse, Komposition, Aggregation
4. Kardinalität
5. Beispiel

# 1 Objekte und Klassen

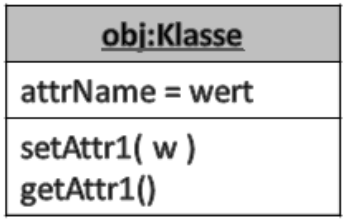
In einem UML Klassendiagramm kann man darstellen:




- **Objekte** (mit klein geschriebenem Namen “obj“, z.B. “erik“, “anja“, “petra“),




- **Klassen** als Zusammenfassung aller Objekte mit gleichen Eigenschaften (Attributen, Methoden). Namen von Klassen werden mit dem ersten Buchstaben groß geschrieben, z.B. Klasse “Student“ (Einzahl).
  - **erik:Student** oder **s837284:Student** drücken Individuen aus.



# 1.1 Objekt, Attribut, Operation

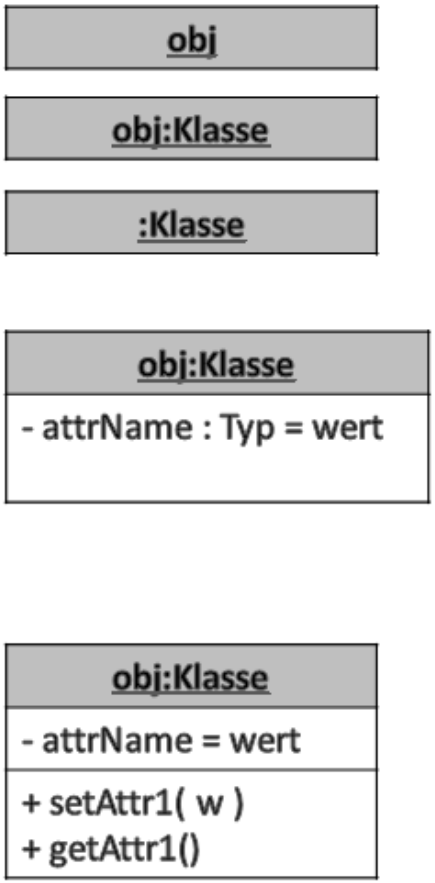
- **Objekt (Instanz)**
  - **obj** – Objektname, klein geschrieben, unterstrichen.
  - **obj:Klasse** – falls Objekt einer Klasse.
  - **:Klasse** – anonymes Objekt.
  - Beispiele: bus1, bus2, :Bus, anton:Fahrer, paul:Fahrer .

- **Attribut**
  - Attribute **sollten** private sein und Werte nur über Operationen ändern.

- **Operation**
  - als Funktionen, Methoden aufrufbar, um Objektzustand zu verändern.

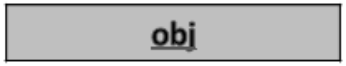
**Sichtbarkeit (visibility)**

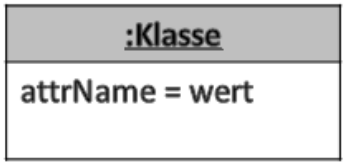
- \+ public
- ~ package
- # protected
- - private

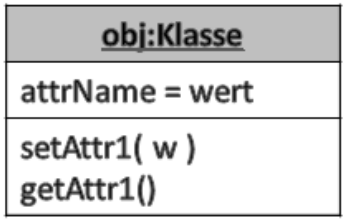


# 1.2 Objekt, Attribut, Operation

- **Objekt:** Repräsentation eines Verwaltungsgegenstandes in einem objektorientierten Modell und später in einem Softwaresystem.
- **Attribut:** Eigenschaft (property) eines Objektes. Die Gesamtheit der Attribute eines Objektes wird auch als Zustand des Objektes bezeichnet.
- **Operation:** Methoden zur Beschreibung des Verhaltens eines Objektes.

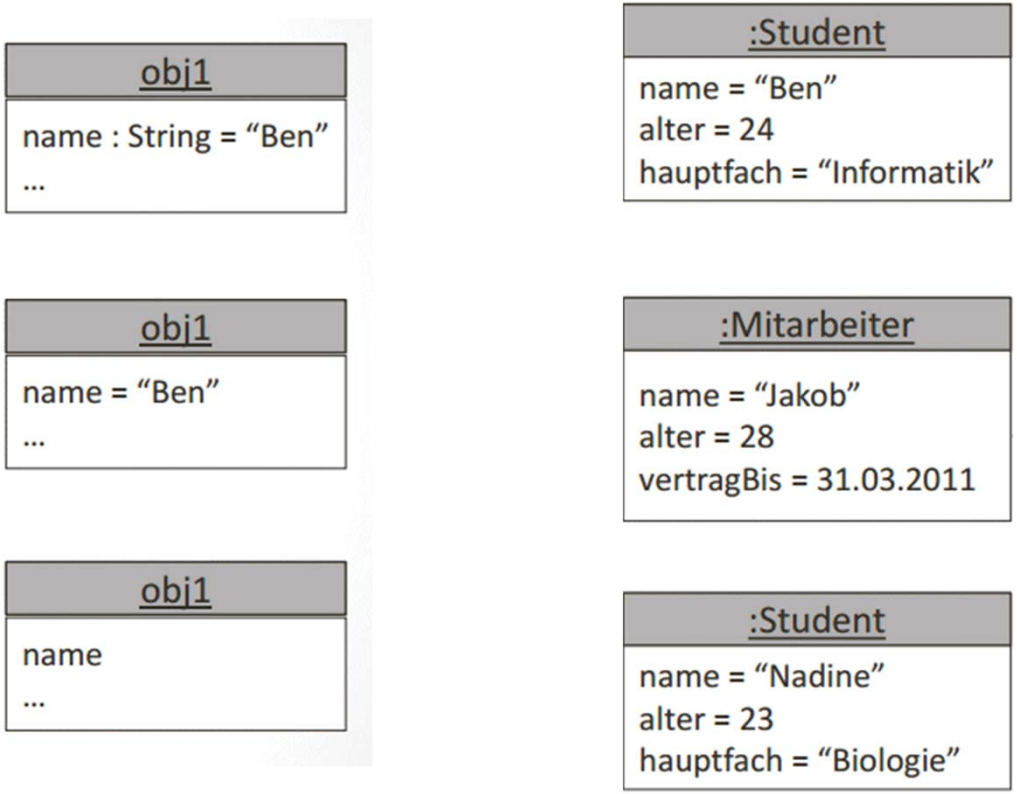


# 1.3 Beispiele: Objekte

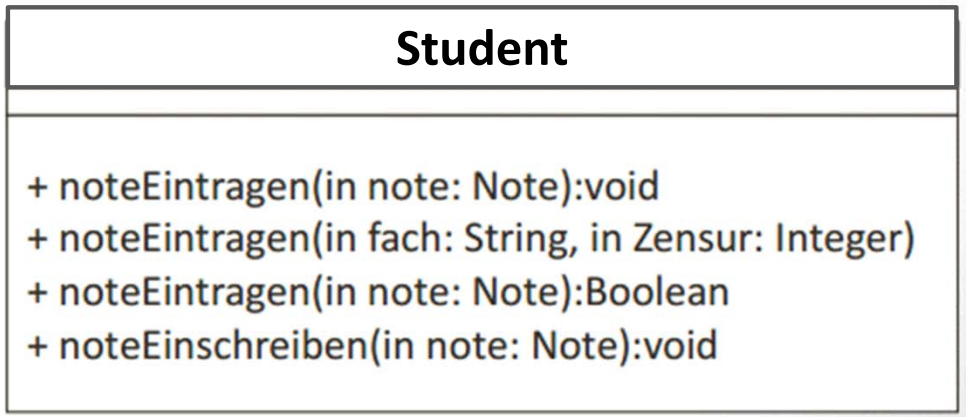
Objekt-Beispiele:



# 1.4 Klassen: Methodennamen

**Eindeutigkeit**

- Operationen müssen in einer Klasse eindeutig sein
- unterscheiden sich in
  - Name der Operation
  - Parameterliste (Typ)
  - Rückgabetyp



# 2 Schnittstellen (Interfaces)

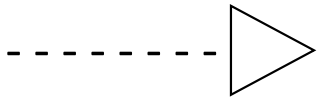
Schnittstellen sind generell Schnitt- oder Verbindungspunkte zwischen Teilen.

- **Schnittstellen in Software** definieren das Verhalten (eines Systems, einer Komponente, der Objekte einer Klasse) über eine Menge von Operationen (Methoden), die an dem Bezugspunkt aufgerufen werden können.
- **Klassen** eignen sich zur Darstellung von Schnittstellen, indem nur Methoden angegeben werden (keine Attribute).
- In UML werden Schnittstellen oft nicht mit dem UML-Symbol für Interfaces dargestellt, sondern als attributlose Klasse mit dem Stereotype *<<interface>>*.

# 2.1 Schnittstellen (Interfaces) in UML

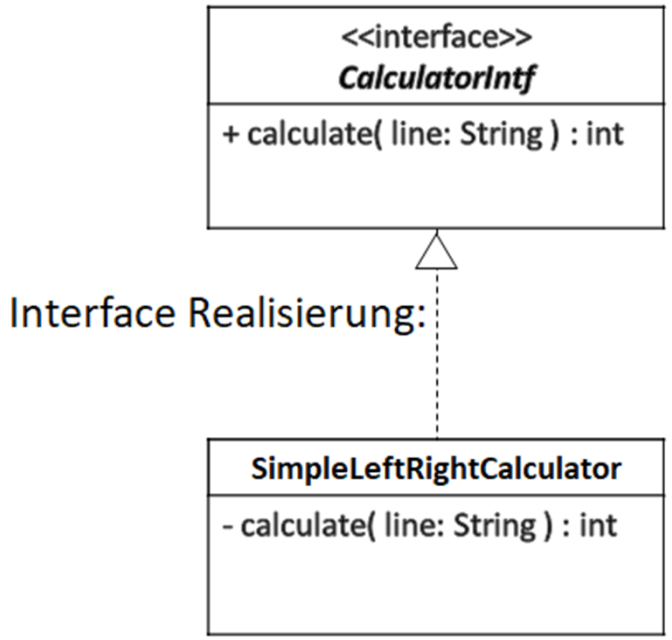
**Interface:** ist eine

Assoziation



zwischen einer Klasse und

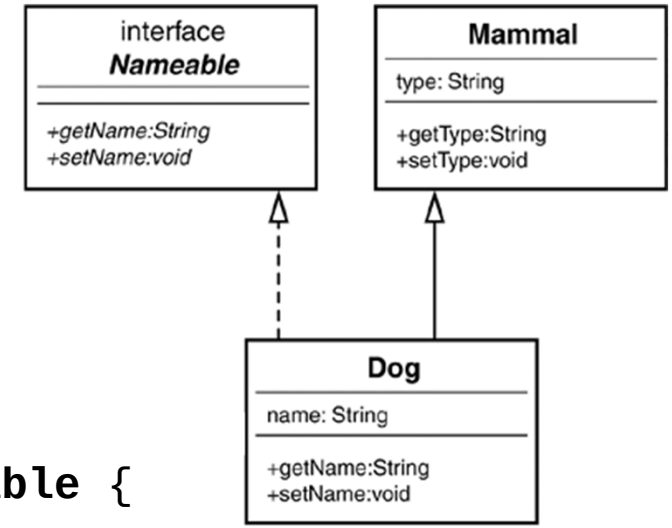
einem Interface.




Bedeutet, dass SimpleLeftRightCalculator alle Methoden des *CalculatorIntf* implementieren muss.

# 2.2 Interface in Java

**Interface:** Klassen können sowohl erben (in Java nur von einer Super-Klasse) als auch (mehrere) Interfaces realisieren.



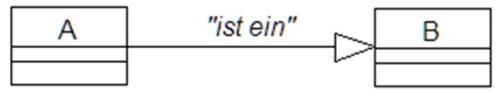

```
class Dog extends Mammal implements Nameable {
  …
}
```

# 3 Beziehungen zwischen Klassen

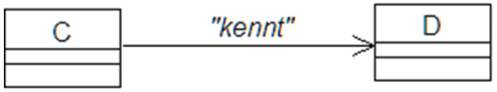
Klassen stehen zu anderen Klassen in Beziehung:

- Eine Klasse „benutzt“ (uses) z.B. eine andere, indem z.B. eine Methode einer anderen Klasse aufgerufen wird oder ein Objekt der anderen Klasse vorkommt.
- Beziehungen zwischen Klassen können selbst Klassen sein, z.B.
  - **:Kunde bestellt :Artikel**. Dann ist **:Bestellung** selbst eine Klasse, die Attribute wie z.B. Artikel-Nr., Artikel-Anzahl, Kunden-Nr., Bestell-Id, Bestelldatum usw. enthält.
- Weiterhin können Gemeinsamkeiten von Klassen in einer übergeordneten Klasse zusammengefasst (“generalisiert“) werden. Eine abgeleitete Klassen „ist“ (is-a) dann eine Spezialisierung dieser Klasse und erbt alle (public und protected) Attribute bzw. Methoden der übergeordneten Klasse.

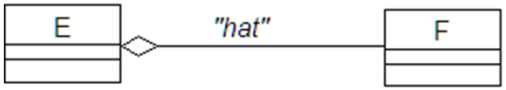
# 3.1 Beziehungen (Assoziationen) in UML



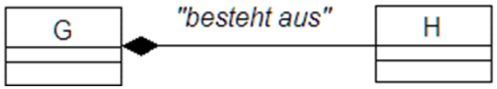

**Spezialisierung / Generalisierung**



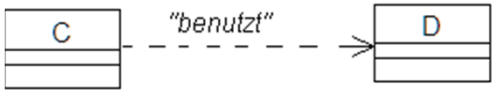

**Assoziation**




**Aggregration**




**Komposition**

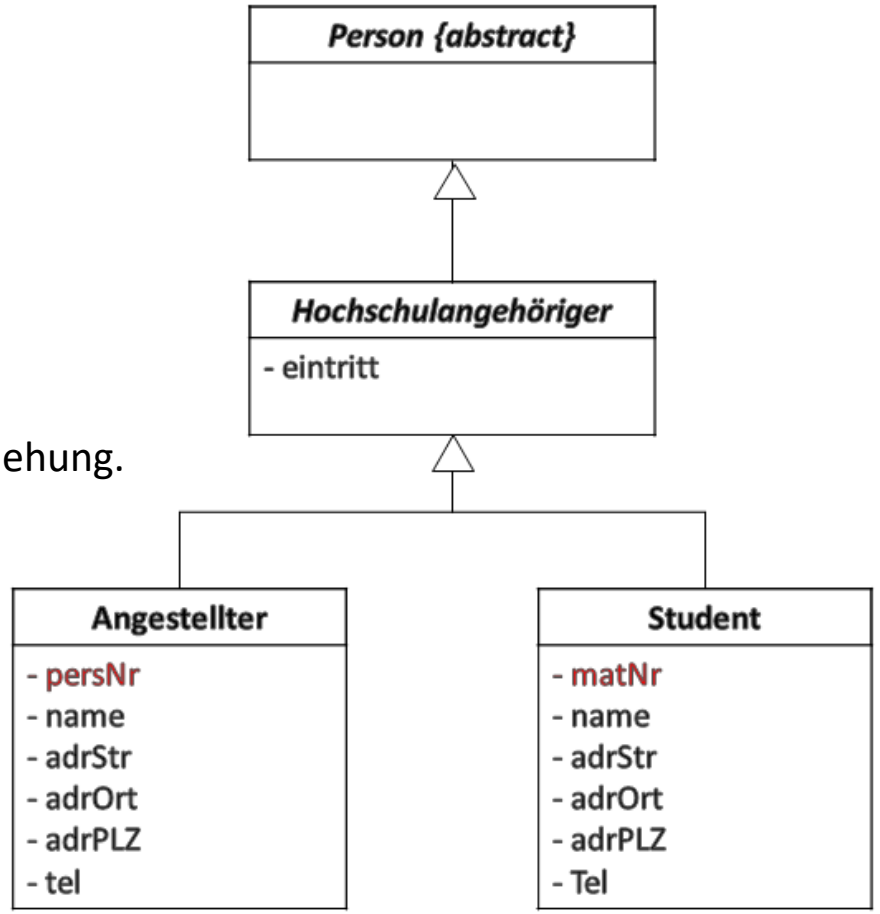



**Dependecy**

# 3.2 Generalisierung in UML

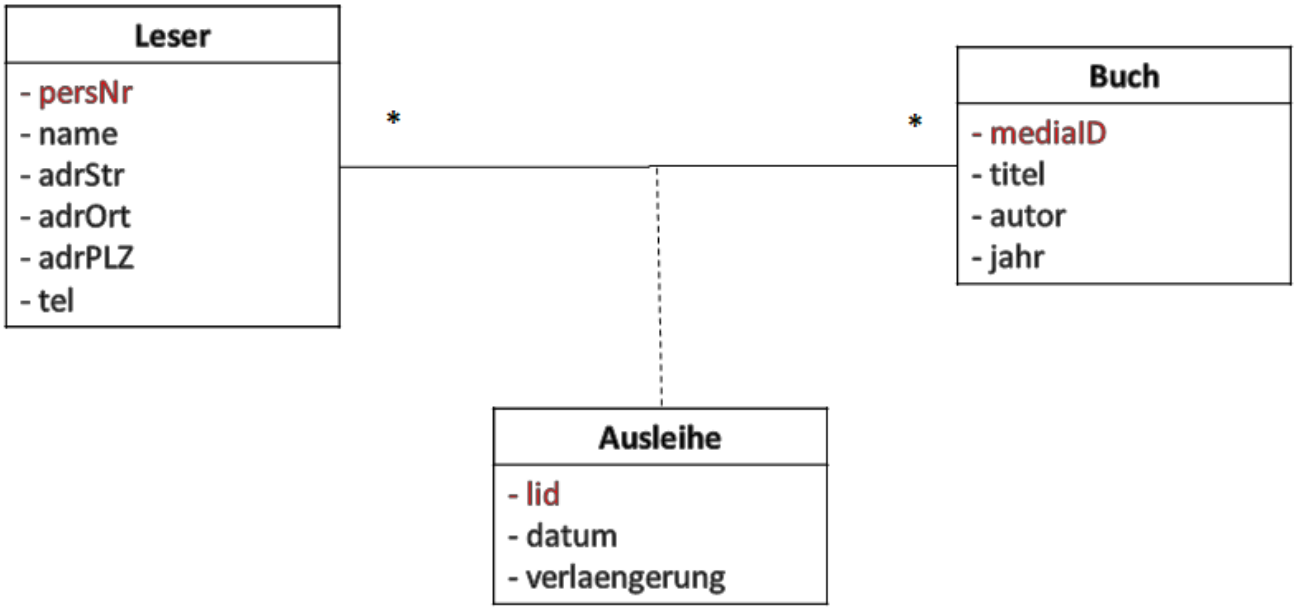
**Generalisierung** von Klassen.

***"ist ein" (is_a)*** – Beziehung.



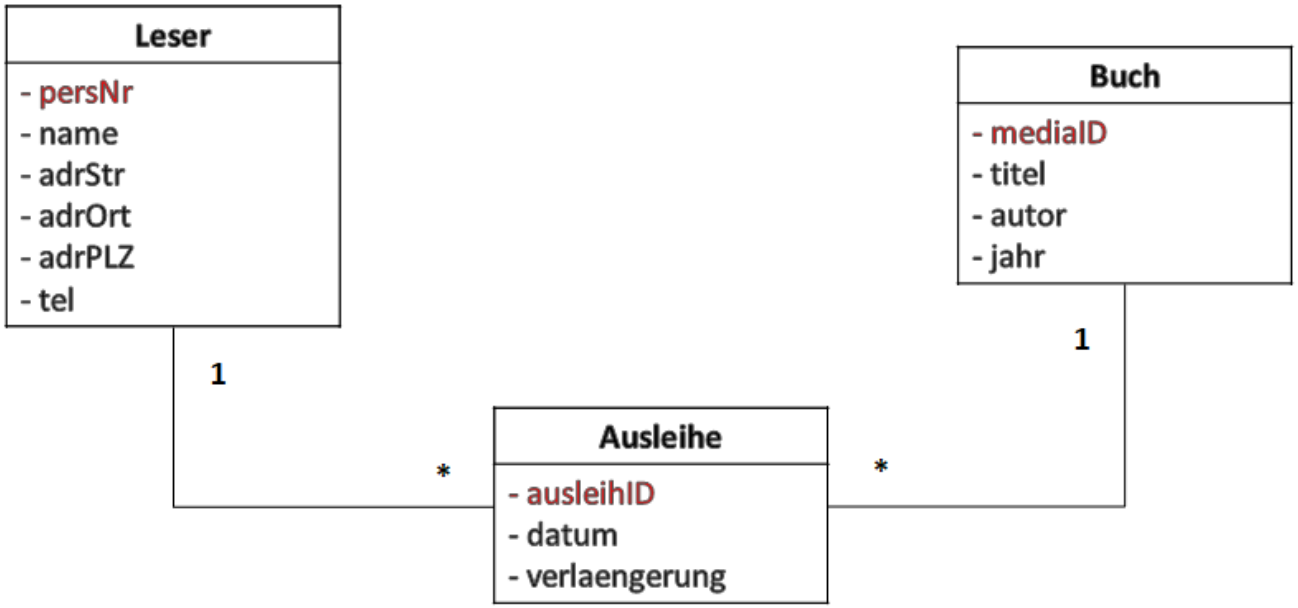
# 3.3 Assoziations-Klasse

**Assoziationsklasse:** besitzt Eigenschaften einer Klasse und einer Assoziation.



# 3.4 Auflösung von Assoziationsklassen

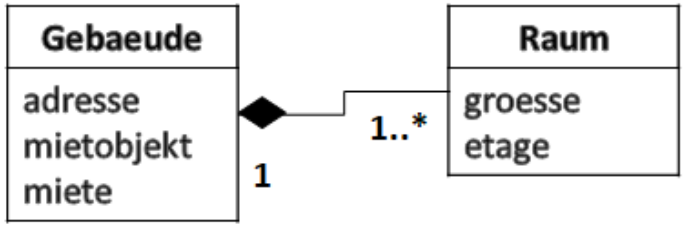
**Assoziationsklasse**: besitzt Eigenschaften einer Klasse und einer Assoziation.



# 3.5 Komposition, Aggregation

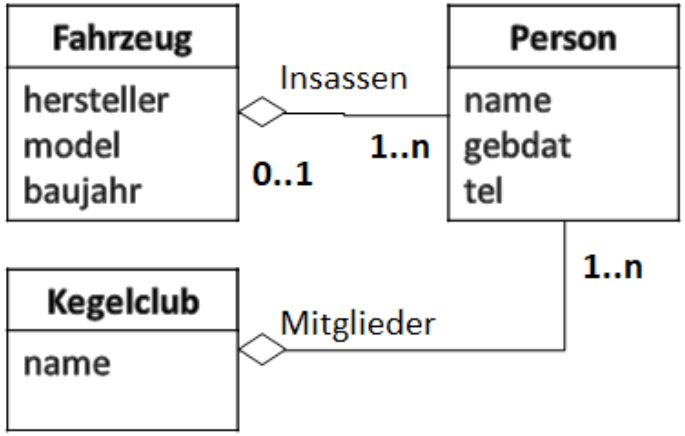
**Komposition:** Drückt Zusammensetzung aus.

Etwas ist ein Bestandteil, das nicht ohne den Aussenrahmen existiert.




***"ist Teil von" (part_of)*** – Beziehung.

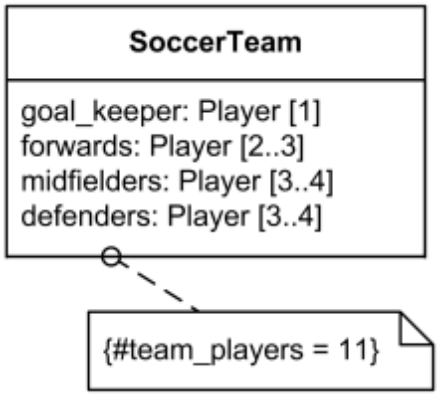
**Aggregation:** Drückt einen Bezug aus, dass etwas dazu gehört, das jedoch unabhängig existiert und auch woanders zugehörig sein kann.

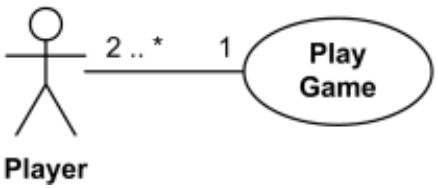


# 4 Kardinalität

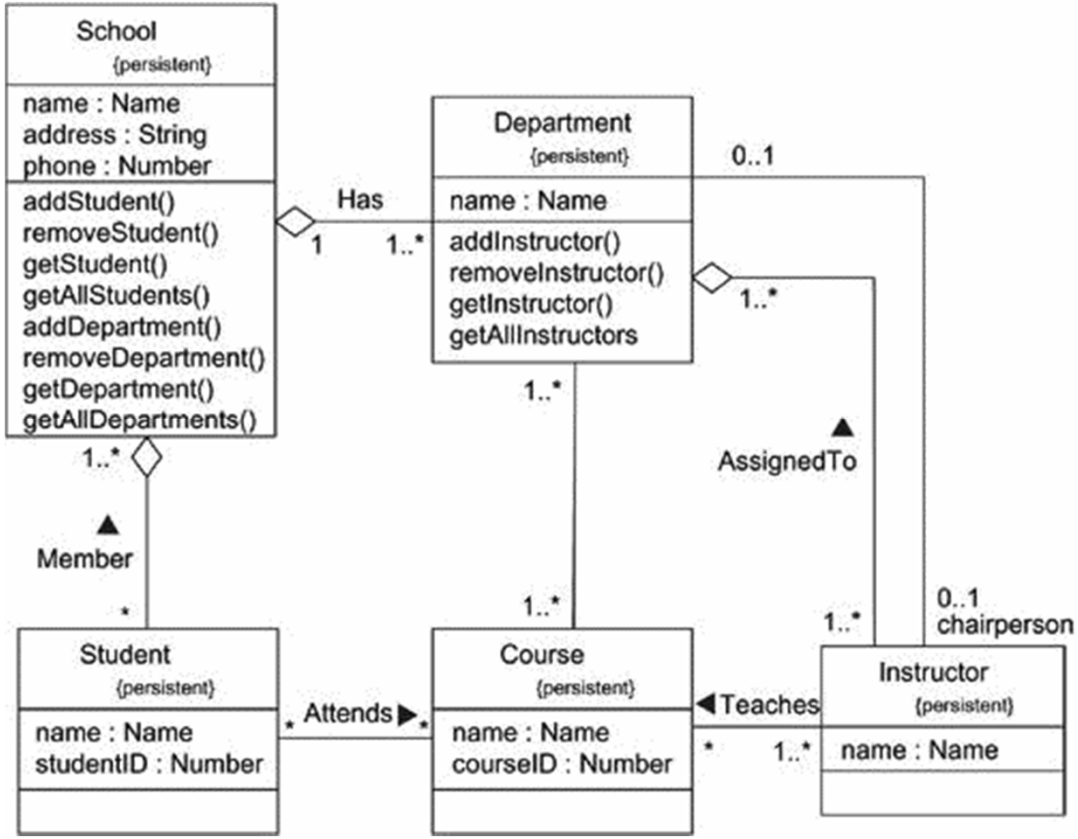
Kardinalität drückt die Anzahl der Elemente einer Beziehung aus.

| Multiplicity | Option | Cardinality |
|---|---|---|
| 0..0 | 0 | Collection must be empty |
| 0..1 | | No instances or one instance |
| 1..1 | 1 | Exactly one instance |
| 0..* | * | Zero or more instances |
| 1..* | | At least one instance |
| 5..5 | 5 | Exactly 5 instances |
| m..n | | At least m but no more than n instances |



# 5 Beispiel – UML Klassen Diagramm




Welche Aussagen können aus diesem Diagramm gelesen werden?

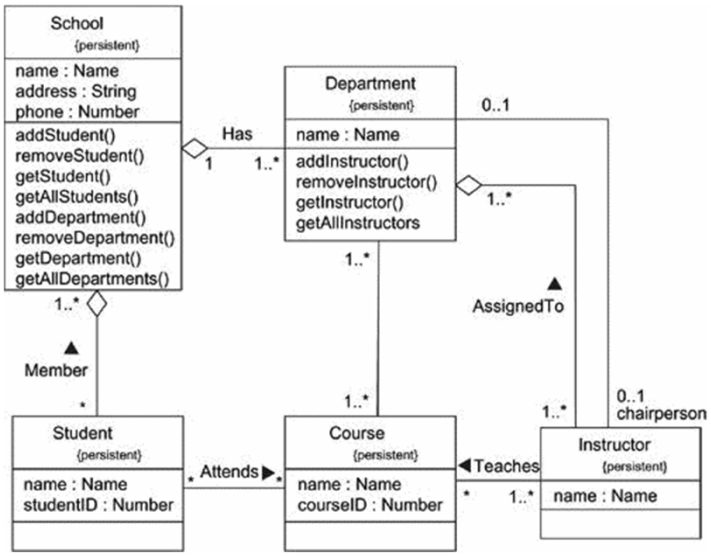
# 5.1 Beispiel – UML Klassen Diagramm

Beispiele richtiger Aussagen:

- Eine Schule [1] besteht aus mehreren [1..*] Departments.
- Ein Department [1] hat mehrere [1..*] Instruktoren.
- Ein Instruktor [1] gehört einem oder mehreren [1..*] Departments an.
- Ein Department [0..1] kann einen Instruktor [0..1] als Chairperson haben.
- Ein Instruktor [0..1] kann Chairperson in nur einem Deptartment [0..1] sein.
- Ein Department gibt mindestens einen [1..*] Kurs.
- Departments [1..*] können Kurse gemeinsam [1..*] halten.
- Alle Klassen sind persistent.

Beispiele falscher Aussagen (nach Diagramm):

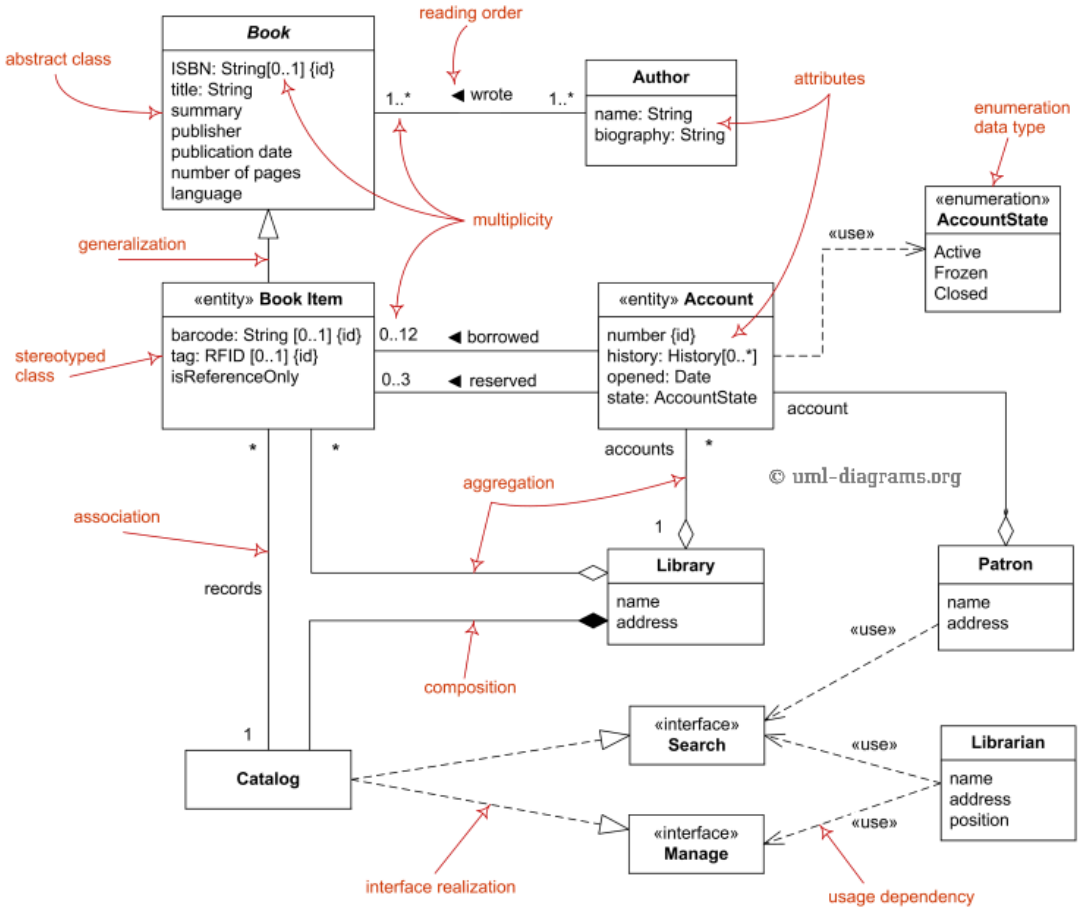
- Jedes Department hat eine Chairperson (falsch, weil [0..1] ).
- Die Schule muss mindestens einen Studenten haben (falsch, weil [*], d.h. [0] ist erlaubt ).
- Ein Student muss mindestens einen Kurs belegen (falsch, weil [*]).



# Zusammenfassung

1. Objekte und Klassen
   - Objekt, Attribute, Methoden
2. Schnittstellen (Interfaces)
3. Beziehungen zwischen Klassen
   - Generalisierung, Assoziationsklasse, Komposition, Aggregation
4. Kardinalitäten
5. Beispiel

# Zusammenfassung: UML Klassendiagramm




Quelle: [http://www.uml-diagrams.org/class-diagrams-overview.html](http://www.uml-diagrams.org/class-diagrams-overview.html).